

FITBUDDY AI

5. Project Development Phase

PROJECT REPORT DOCUMENT

Project: FitBuddy AI — Fitness Plan Generator

Group 1

M. Vijayasathy — Team Leader

Bharath

Harsha

Sumith

Arun Vasan

Objective

To describe the implementation of the FitBuddy AI application and the steps required to build and run it.

Prepared for academic project submission

5. Project Development Phase

Technology Stack

• Python 3.11+, FastAPI, Jinja2 templates, SQLAlchemy ORM, SQLite database, Pydantic validation, Google GenAI SDK, python-dotenv, Uvicorn and pytest/httpx.

Project Structure

- app/main.py — application entry point and startup database initialization.
- app/routes.py — HTML routes and REST API.
- app/ai_service.py — prompt construction, Gemini generation, mock mode and formatting.
- app/schemas.py — validated request/response models.
- app/database.py and app/crud.py — persistence layer.
- templates/ — base, home, result, error and admin pages.
- static/css/styles.css — responsive styling.
- tests/test_app.py — automated tests.
- .env.example, requirements.txt and README.md — configuration and setup.

Development Workflow

- 1. Create Python virtual environment. 2. Install dependencies from requirements.txt. 3. Copy .env.example to .env. 4. Add Gemini API key for live generation or keep mock mode enabled for local testing. 5. Configure secure admin credentials. 6. Start the server using Uvicorn. 7. Verify home page, health endpoint and plan generation.

Core Implementation

- Profile form collects user ID, name, age, weight, goal, intensity and experience level. Pydantic validates supported values. The application stores the profile, requests a structured seven-day plan and a concise wellness tip, saves the plan and renders a result page.

AI Integration

- The AI service builds prompts using the user's profile and requests a JSON structure with an overview and exactly seven day objects. The wellness prompt asks for a brief general tip. Feedback is passed with the current plan to request a safe revision. The application supports mock responses when no live API is configured and mock mode is enabled.

Database Implementation

- SQLAlchemy models represent users and plans. New plans mark earlier plans for that user as not current. Revised plan text and user feedback are saved to the current plan record. SQLite is the default local database.

Run Commands

- Windows: `python -m venv .venv ; .venv\Scripts\Activate.ps1 ; pip install -r requirements.txt ; uvicorn app.main:app --reload`
- macOS/Linux: `python3 -m venv .venv ; source .venv/bin/activate ; pip install -r requirements.txt ; uvicorn app.main:app --reload`
- Open <http://127.0.0.1:8000>. API documentation: <http://127.0.0.1:8000/docs>. Run tests with `pytest -q`.

Security & Configuration

- Store GEMINI_API_KEY in .env, never commit secrets, set unique admin credentials and use HTTPS plus stronger deployment controls before public hosting. The default configuration is intended for development, not production.

Project Team — Group 1

- M. Vijayarathy — Team Leader
- Bharath
- Harsha
- Sumith
- Arun Vasan